VISOR ? Visual Interlock Smart Operation and Response
Complete Technical Project Report

Version: 2.0 | **Date:** September 2026
Classification: For Academic Publication and Patent Filing

Table of Contents

1. Executive Summary
2. Problem Statement and Motivation
3. System Architecture Overview
4. Subsystem 1 ? Smart Helmet Authentication (Edge AI)
5. Subsystem 2 ? Physical Sensor Interlock (Helmet ESP32)
6. Subsystem 3 ? Wireless Communication Layer (ESP-NOW)
7. Subsystem 4 ? BLE Gateway and Motor Interlock
8. Subsystem 5 ? YOLOv3-Lite Custom Model (ML Pipeline)
9. Subsystem 6 ? Cloud Backend (VISOR Server)
10. Subsystem 7 ? Flutter Mobile/Dashboard Application
11. Subsystem 8 ? ESP32 Rider GPS Tracker
12. AprilTag Generation and Physical Manufacturing
13. Complete Data Flow and System Integration
14. All Formulas and Mathematical Models
15. All Key Parameters, Thresholds and Constants
16. Hardware Bill of Materials
17. Software Dependencies and Versions
18. Performance Summary and Test Results
19. Novel Contributions and Patent Claims
20. Conclusion and Future Work
21. Appendices

---

1. Executive Summary

VISOR (Visual Interlock Smart Operation and Response) is an integrated smart transportation safety system for electric vehicle (EV) riders and emergency responders. It solves two critical road-safety problems:

Problem A ? Helmet Compliance Enforcement:
Existing vision-only systems can be physically bypassed. VISOR enforces multi-factor authentication requiring SIMULTANEOUS confirmation from:
- Vision AI: YOLOv2 face detection + AprilTag fiducial geometric check (K210 KPU)
- Physical pressure: FSR sensor embedded in helmet interior
- Sobriety: MQ-3 alcohol detector mounted in helmet

Only when all three confirm correct usage does the vehicle operate at full speed.

Problem B ? Ambulance Path Clearance:
EV riders cannot hear sirens. VISOR provides real-time proximity warnings using cloud-connected GPS. When an ambulance activates emergency mode, EV riders AHEAD within 500 m receive:
- Full-screen cinematic HUD on dashboard

- Voice TTS announcement
- Haptic vibration patterns
- MQTT-triggered hardware buzzer

The system spans eight subsystems: Edge AI (K210, 28-30 FPS, 0.3 W), ESP-NOW + BLE wireless, Socket.IO cloud, Flutter dashboard, and a custom YOLOv3-Lite training pipeline.

Demonstrated at Smart India Hackathon 2026.

---

2. Problem Statement and Motivation

India accounts for ~11% of global road accident fatalities (WHO 2023). Two-wheeler riders constitute ~44% of road deaths. Non-helmet usage and alcohol-impaired riding are leading factors. Emergency vehicles lose 30-40% of critical response time due to traffic not yielding.

Problem	Existing Solutions	VISOR Improvement
Helmet detection	Vision-only camera	Vision + pressure + AprilTag interlock
Alcohol detection	Standalone breathalyzers	Inline MQ-3 in ignition interlock
Ambulance clearance	Siren + traffic signals	GPS corridor tracking, 500m ahead-only alert
Alert modality	None for EV riders	HUD + TTS + vibration + hardware buzzer

---

3. System Architecture Overview

```
+-------------------------+  UART 115200 baud  +----------------------+
| Sipeed Maixduino (K210) |------------------->| Internal ESP32       |
| - YOLOv2 face detection |  TAG:1/0 ID X Y\n  | - Parse K210 UART    |
| - AprilTag TAG36H11 det.|<-------------------| - ESP-NOW broadcaster|
| - OV2640 camera QVGA    |  SEND_OK/FAIL      +----------+-----------+
+-------------------------+                               |
                                                  ESP-NOW | 2.4 GHz
+-------------------------+      ESP-NOW                   |
| Helmet ESP32            |-------------------------->     |
| - MQ-3 Alcohol GPIO34   |                      +---------v-----------+
| - FSR Pressure GPIO35   |                      | Receiver ESP32      |
| - Hall Effect GPIO32    |                      | - ESP-NOW dual-source|
+-------------------------+                     | - BLE GATT server    |
                                                | - L298N motor driver |
                                                +----------+-----------+
                                                           | BLE 4.2 GATT
Notify
                                                +----------v-----------+
                                                | Flutter App          |
                                                | - EV Dashboard       |
                                                | - Ambulance Interface|
```

```
                                                  | - BLE + GPS + Socket  |
                                                  +----------+-----------+
                                                             | Socket.IO
WebSocket
                                                  +----------v-----------+
                                                  | VISOR Cloud (Railway)  |
                                                  | - Express + Socket.IO |
                                                  | - MQTT Bridge         |
                                                  | - Priority Engine     |
                                                  | - MongoDB + Firebase  |
                                                  +----------+-----------+
                                                             | MQTT QoS 1
                                                  +----------v-----------+
                                                  | ESP32 GPS Tracker     |
                                                  | - NEO-6M GPS          |
                                                  | - MQTT pub/sub        |
                                                  | - OLED + Buzzer       |
                                                  +-----------------------+
```

Communication Protocol Summary

Link	Protocol	Speed	Direction
K210 to Internal ESP32	UART Serial2	115200 baud	TX only
Internal to Receiver ESP32	ESP-NOW 2.4GHz	~1 Mbps	Unidirectional
Helmet to Receiver ESP32	ESP-NOW 2.4GHz	~1 Mbps	Unidirectional
Receiver to Flutter	BLE 4.2 GATT Notify	~250 Kbps	Bidirectional
Flutter to Cloud	Socket.IO WebSocket	Internet	Bidirectional
Cloud to GPS Tracker	MQTT QoS 1 HiveMQ	Internet	Bidirectional

---

4. Subsystem 1 ? Smart Helmet Authentication (Edge AI)

4.1 Hardware Platform: Sipeed Maixduino (K210)

Chip: Kendryte K210 dual-core 64-bit RISC-V at 400 MHz
KPU: Dedicated MAC array, 1 TOPS at INT8, 2 MB SRAM
Camera: OV2640 via DVP interface
Display: 2.4" ILI9341 TFT LCD 320x240 RGB565
UART: IO6 (RX) / IO7 (TX) ? UART1 ? wired to ESP32 GPIO3/GPIO1
Runtime: MicroPython (MaixPy firmware)

Camera Configuration:
```python
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)    # 320 x 240
sensor.set_contrast(2)               # Maximum contrast for low light
sensor.set_brightness(1)
sensor.set_auto_gain(True)
sensor.set_gainceiling(8)            # Up to 8x gain
sensor.set_auto_exposure(True)
sensor.set_auto_whitebal(True)
sensor.run(1)
```

Files: sipeed/main.py (60s state machine), deploy/main.py (deploy version)

4.2 YOLOv2 Face Detection on KPU

Model: face.kmodel ? pre-trained YOLOv2 face detector
Storage: /sd/face.kmodel (SD card) or SPI Flash at 0x300000
Input: 320x240 QVGA RGB565, KPU latency ~3.5 ms

YOLOv2 Anchors (5 pairs for face, in grid-cell units):
```python
ANCHORS = (1.889, 2.5245, 2.9465, 3.94056, 3.99987, 5.3658,
           5.155437, 6.92275, 6.718375, 9.01025)
```

Detection Parameters:
```
CONF_THRESHOLD = 0.18    # Minimum objectness x class confidence
NMS_THRESHOLD  = 0.30    # Non-Maximum Suppression IoU threshold
```

FORMULA 1 ? EMA Bounding Box Smoothing:
```
x[t] = alpha * x_raw[t] + (1 - alpha) * x[t-1]
alpha = 0.15 (sipeed/main.py) or 0.30 (deploy/main.py)
Applied to all 4 dimensions: x, y, w, h
```

Eliminates jitter between frames. With alpha=0.15 (heavy smoothing), the box is very stable.

Ghost Frame Persistence (MAX_LIFETIME):
When face is not detected, last box held for MAX_LIFETIME frames before expiry:
```python
if best_det is not None:
    face_lifetime = MAX_LIFETIME   # 10 or 25 frames
else:
    face_lifetime -= 1
    if face_lifetime <= 0:
        last_face = None
```

Rolling History Debounce:
```python
HISTORY_LEN = 12; HISTORY_HIT = 2
face_history.append(face_seen)
if len(face_history) > HISTORY_LEN: face_history.pop(0)
face_seen_stable = face_history.count(True) >= HISTORY_HIT   # 2/12 threshold
```

4.3 AprilTag Visual Fiducial Detection

Tag Family: TAG36H11 (10x10 matrix, 6x6 data bits, robust Hamming error correction)

Tag ID: 0 (binary code: 0xd5d628584)
Detection API: image.find_apriltags(roi=..., families=image.TAG36H11)
Physical Size: 4.23 cm (500px at 300 DPI)
Placement: Front of helmet at chin/visor level

Memory Constraint:
find_apriltags() allocates ~12 bytes/pixel internally. Full QVGA = 76,800px =
~921 KB ? exceeds K210's 2 MB. Solution: use tight ROI below face box.

FORMULA 2 ? Dynamic ROI Calculation:
```
roi_x = max(0, face_left_x - 25)
roi_y = max(0, face_bottom_y)
roi_w = min(face_right_x - face_left_x + 50, 320 - roi_x)
roi_h = min(240 - roi_y, 120)         [120 rows max ? memory safe]
Typical ROI allocation: ~14,400 px = ~173 KB ? fits easily
```

Eye-Region Sub-Box (top 45% of full face height):
```python
eye_fy = fy
eye_fh = max(10, int(fh * 0.45))    # Top 45%
face_bottom_y = fy + eye_fh          # AprilTag MUST be below this line
```

Tag Validation:
```python
if tcy > face_bottom_y:   # Tag Y-center below eye threshold
    tag_detected = True   # Geometric position confirmed
```

4.4 Dual-Factor Visual Interlock Logic

```
HELMET_WORN = face_seen_stable AND tag_seen
```

Why both are required:
- Face alone: bypassable with photograph
- Tag alone: bypassable by holding tag near camera
- Both: tag BELOW face AND within face column -> tag physically at chin ->
helmet being worn

Camera View Geometry (320x240):
```
+-----------------------------------------------+
|                                               |
|  +-----------------------------+              |
|  |    EYE REGION BOX (45%)     |  <- Green or Red box
|  |    face_left_x to right_x   |
|  +-----------------------------+              |
| ============= YELLOW LINE ================== |
|  (AprilTag center Y must be BELOW this)      |
|  +-----------------------------+              |
|  | [TAG36H11] ID=0  (Green)    |  <- Tag detected
|  +-----------------------------+              |
```

```
+-----------------------------------------------+
```

4.5 State Machine ? 60-Second Cycle (sipeed/main.py)

```
TOTAL CYCLE = 60 seconds

Phase 1 (0-30s): STATE_SCANNING
  Blue LED ON, scan for face + tag every frame
  HELMET_WORN confirmed? -> Lock to PASS
  30s timeout without PASS? -> Lock to FAIL

Phase 2 (30-60s): STATE_RESULT_LOCKED
  PASS: Green LED ON, Buzzer OFF, send TAG:1 every frame
  FAIL: Red LED ON, Buzzer ON, send TAG:0 every frame
  No re-scanning during this phase

Phase 3: New cycle restarts
```

State	Blue LED	Green LED	Red LED	Buzzer
STATE_SCANNING	ON	OFF	OFF	OFF
RESULT PASS	OFF	ON	OFF	OFF
RESULT FAIL	OFF	OFF	ON	ON

4.6 UART Protocol (K210 to Internal ESP32)

Config: UART1, 115200 baud, 8N1, LF-terminated ASCII

Condition	Payload
Helmet WORN, tag visible	TAG:1 ID:0 X:162 Y:203\n
Helmet WORN, state held	TAG:1 ID:0 X:-1 Y:-1\n
Helmet NOT WORN	TAG:0\n

ACK from ESP32 to K210:
```
SEND_OK\n    <- ESP-NOW packet delivered
SEND_FAIL\n  <- ESP-NOW failed
```

ESP32 Parser accepted not-detected strings:
"TAG:0", "Helmet Not Detected", "HELMET NOT DETECTED", "NO HELMET", "Not Detected"

---

5. Subsystem 2 ? Physical Sensor Interlock (Helmet ESP32)

File: helmet_esp32/helmetesp/helmetesp.ino | **Board:** ESP32-WROOM-32

5.1 MQ-3 Alcohol Sensor with Hysteresis

Pin: GPIO 34 (ADC1_CH6), 12-bit (0-4095)

Warmup: 120 seconds firmware-gated
Threshold: ALCOHOL_THRESHOLD = 1900 ADC counts

FORMULA 3 ? Alcohol Hysteresis:
```
ENTER alert: rawAlcohol > 1900
EXIT  alert: rawAlcohol < 1900 - 100 = 1800
Dead-band = 100 counts (prevents threshold oscillation)
```

```c
if (!alcoholAlert) {
    if (rawAlcohol > ALCOHOL_THRESHOLD) alcoholAlert = true;
} else {
    if (rawAlcohol < (ALCOHOL_THRESHOLD - 100)) alcoholAlert = false;
}
```

Warmup Gate:
```c
if (millis() > 120000) rawAlcohol = analogRead(ALCOHOL_PIN);
else rawAlcohol = 0;   // Force zero during warmup
```

5.2 Force Sensitive Resistor (FSR) ? Head Presence

Pin: GPIO 35 (ADC1_CH7), 12-bit
Wiring: VCC 3.3V -> FSR -> GPIO35 -> 10kOhm pull-down -> GND

FORMULA 4 ? FSR Voltage Divider:
```
V_out = V_cc * R_pulldown / (R_FSR + R_pulldown)
ADC   = 4095 * R_pulldown / (R_FSR + R_pulldown)
As head pressure increases: R_FSR decreases -> V_out increases -> ADC
increases
```

Thresholds:
```c
FSR_TOUCH_THRESHOLD      = 80     // Slight touch = PASS
FSR_REMOVE_THRESHOLD     = 25     // Near-zero = definitive removal
FSR_HOLD_MS              = 6000   // 6-second PASS hold window
FSR_REMOVAL_DEBOUNCE_MS  = 500    // Removal must persist 500ms for FAIL
```

5.3 Hall Effect Strap Lock Sensor

Pin: GPIO 32 (ADC1_CH4) | **Threshold:** HALL_THRESHOLD = 150 (deviation)

FORMULA 5 ? Hall Baseline Calibration:
```
baseline  = (1/20) * SUM(ADC_i, i=1..20)   [20 samples at boot, 10ms apart]
deviation = |ADC_current - baseline|
strapLocked = (deviation < 150)
```

Current deployment: Hall effect BYPASSED (strapLocked = true always ? for field testing)

Combined Worn Condition:
```c
bool fsrAlert = fsrActive && strapLocked;
```

5.4 FSR State Machine with Debounce

```
IDLE (holdingPass=false, fsrActive=false)
  rawFsr >= 80 (TOUCH)?  YES -> PASS_HOLD, start 6s timer
  NO  -> stay IDLE

PASS_HOLD (holdingPass=true, fsrActive=true)
  Early-FAIL: rawFsr < 25 (REMOVE) for >= 500ms continuously?
    YES -> EARLY FAIL (holdingPass=false, fsrActive=false)
    NO  -> cancel debounce, stay PASS
  Hold expiry: millis() - fsrDetectTime >= 6000?
    YES: rawFsr >= 80? YES -> restart 6s hold  NO -> FAIL
    NO  -> continue PASS

FAIL -> back to IDLE
```

LED indicator:
```c
if (alcoholAlert || !fsrAlert) { RED = HIGH; GREEN = LOW; }   // Unsafe
else                           { RED = LOW;  GREEN = HIGH; }  // Safe
```

Transmission: Every 500ms via ESP-NOW

---

6. Subsystem 3 ? Wireless Communication (ESP-NOW)

Protocol: IEEE 802.11 connectionless, 2.4 GHz, Channel 1, no encryption
Max payload: 250 bytes | **Latency:** <10 ms | **Range:** ~100 m LoS

6.1 Internal ESP32 Sender (internal_esp32/internal_esp32.ino)

Target MAC: 28:05:A5:E2:C6:64 (Receiver ESP32 Station MAC)

Serial2 for K210 reading:
```c
Serial2.begin(115200, SERIAL_8N1, 3, 1);
// GPIO3 = RX from K210 IO7 | GPIO1 = TX to K210 IO6
```

Heartbeat (HEARTBEAT_TIMEOUT_MS = 2000 ms):
```c
if (millis() - lastK210RxTime > 2000) {
    sendToExternal(false, -1, -1, -1);  // Keepalive not-detected
    lastK210RxTime = millis();
```

```
}
```

ESP-NOW v2/v3 compatibility:
```c
#if defined(ESP_ARDUINO_VERSION) && (ESP_ARDUINO_VERSION >=
ESP_ARDUINO_VERSION_VAL(3, 0, 0))
  #define ESP_NOW_CORE_V3
#endif
// OnDataSent callback signature differs between v2 and v3
```

6.2 Receiver ESP32 Hub (receiveresp/receiveresp.ino)

Packet type differentiation by size (len):
```c
if (len == sizeof(tag_message))    -> AprilTag data (Internal ESP32)
if (len == sizeof(helmet_message)) -> Sensor data (Helmet ESP32)
```

Link timeout detection:
```c
const unsigned long LINK_TIMEOUT_MS = 5000;
if (millis() - lastPacketTime > LINK_TIMEOUT_MS) linkLost = true;
```

6.3 ESP-NOW Packet Structures

tag_message ? 20 bytes:
```c
typedef struct {
    bool     detected;    // true if AprilTag detected
    int      tagID;       // Tag ID (-1 = none)
    int      centerX;     // X pixel coordinate (-1 if none)
    int      centerY;     // Y pixel coordinate (-1 if none)
    uint32_t timestamp;   // millis() at send time
} tag_message;
```

helmet_message ? 16 bytes:
```c
typedef struct {
    int      alcoholValue;    // Raw MQ-3 ADC (0-4095)
    int      fsrValue;        // Raw FSR ADC (0-4095)
    bool     alcoholAlert;    // true if alcohol > threshold
    bool     fsrAlert;        // true if helmet worn
    uint32_t timestamp;       // millis() at send time
} helmet_message;
```

---

7. Subsystem 4 ? BLE Gateway and Motor Interlock

7.1 BLE GATT Server

Name: ESP32_HELMET_RECEIVER
Service UUID: 12345678-1234-1234-1234-1234567890AB

Characteristic	UUID	Properties	Data
Helmet	BEB5483E-36E1-4688-B7F5-EA07361B26A8	Read+Notify	0 or 1
Strap	CED11C71-7030-4C10-90D1-64582E5507E8	Read+Notify	0 or 1
Speed	DFB5483E-36E1-4688-B7F5-EA07361B26A8	Read+Notify	km/h uint8

Auto-restart advertising:
```c
void onDisconnect(BLEServer* pServer) {
    deviceConnected = false;
    pServer->startAdvertising();   // Immediate restart
}
```

7.2 Motor Speed Control (L298N)

Pins: MOTOR_IN1=GPIO12, MOTOR_IN2=GPIO13, MOTOR_ENA=GPIO14 (PWM)

```c
const int SPEED_LIMITED = 100;  // ~20 km/h (safety conditions NOT met)
const int SPEED_MAXIMUM = 230;  // ~46 km/h (all conditions PASS)
uint8_t speedKmh = (currentMotorPWM == SPEED_MAXIMUM) ? 46 : 20;
```

7.3 Combined Safety Logic

```
riderNotWearing = (NOT tag_detected) OR (NOT fsrAlert)
alcoholDetected = alcoholAlert

SAFE = (NOT linkLost) AND (NOT helmetLinkLost)
   AND (NOT riderNotWearing) AND (NOT alcoholDetected)

Motor PWM = SPEED_MAXIMUM if SAFE else SPEED_LIMITED
```

Condition	Alert	Motor
Not wearing + Alcohol	"Rider not wearing helmet + Alcohol detected"	LIMITED
Not wearing only	"Rider not wearing helmet"	LIMITED
Alcohol only	"Alcohol detected"	LIMITED
All clear	"speed lock open have a safe ride"	MAXIMUM

---

8. Subsystem 5 ? YOLOv3-Lite Custom Model (ML Pipeline)

Directory: FACE Detection (eye) + AprilaTag Detection/

8.1 Model Architecture ? MobileNetV2-Lite Backbone

File: model/yolov3_lite.py | **Framework:** TensorFlow 2.x / Keras

Input: (160, 224, 1) ? Grayscale | **Total Parameters:** ~398,000

MobileNetV2 Bottleneck Block:
```
Phase 1 Expansion: Conv2D(in_ch * expansion, 1x1) -> BN -> ReLU6
Phase 2 Depthwise: DepthwiseConv2D(3x3, stride=s) -> BN -> ReLU6
Phase 3 Projection: Conv2D(out_ch, 1x1) -> BN
Residual: if stride==1 AND in_ch==out_ch: output = input + projection
```

FORMULA 6 ? ReLU6 Activation:
```
ReLU6(x) = min(max(0, x), 6)
Capped at 6 for INT8 quantization compatibility
```

Backbone Layer Table:

Block	In Shape	Out Shape	Stride	Expansion	Out Ch
Conv Init	160x224x1	80x112x16	2	-	16
BN1	80x112x16	80x112x16	1	1	16
BN2	80x112x16	40x56x24	2	6	24
BN3	40x56x24	40x56x24	1	6	24
BN4	40x56x24	20x28x32	2	6	32
BN5	20x28x32	20x28x32	1	6	32
BN6	20x28x32	10x14x64	2	6	64 (feat_s16)
BN7	10x14x64	10x14x64	1	6	64
BN8	10x14x64	5x7x128	2	6	128 (feat_s32)
BN9	5x7x128	5x7x128	1	6	128

8.2 YOLO Detection Head ? Two-Scale Output

Output channels: 3 anchors * (4 coords + 1 objectness + 1 class) = 18 per cell

Scale 1 (Stride 16): 10x14 grid -> 10x14x18 -> reshape to (2520,)
Scale 2 (Stride 32): 5x7 grid -> 5x7x18 -> reshape to (630,)
Final output: Concatenate -> (3150,) single vector for K210 KPU

Feature Pyramid Network (FPN) for Scale 1:
```
feat_s32 -> Conv2D(64,1x1) -> BN -> ReLU6 -> UpSampling2D(2x2)
                                                    |
                                          Concatenate + feat_s16
                                                    |
                                          Conv2D(64,1x1) -> BN -> ReLU6
                                          Conv2D(128,3x3) -> BN -> ReLU6
                                          Conv2D(18,1x1) -> output_s16
```

8.3 Loss Function Formulation (model/train.py)

FORMULA 7 ? Complete YOLO Loss:
```
L_coord = 5.0 * SUM[obj_mask * (MSE(tx) + MSE(ty) + MSE(tw) + MSE(th))]
```

```
L_conf  = SUM[obj_mask * BCE(conf)] + 0.5 * SUM[noobj_mask * BCE(conf)]
L_class = SUM[obj_mask * BCE(class)]
L_total = (L_coord + L_conf + L_class) / batch_size

Applied to both scales: L_final = L_s16 + L_s32
Coord weight = 5.0 (high accuracy on box positions)
Background weight = 0.5 (downweight empty cells)
```

FORMULA 8 ? YOLO Ground Truth Box Encoding:
```
gx = int(cx * grid_w);    gy = int(cy * grid_h)
tx = cx * grid_w - gx;    ty = cy * grid_h - gy  [cell offsets 0..1]
tw = log(w_px / anchor_w); th = log(h_px / anchor_h) [log-scale from anchor]
```

FORMULA 9 ? YOLO Box Decoding at Inference:
```
bx = sigmoid(tx) + grid_col
by = sigmoid(ty) + grid_row
bw = anchor_w * exp(tw)
bh = anchor_h * exp(th)
conf  = sigmoid(t_o)
class = sigmoid(t_c)
where sigmoid(x) = 1 / (1 + exp(-x))
```

Training Config:
```python
optimizer  = Adam(learning_rate=0.0001)
epochs     = 10
batch_size = 6
EarlyStopping(patience=3), ReduceLROnPlateau(factor=0.5, patience=2)
```

8.4 K-Means Anchor Box Clustering (model/kmeans_anchors.py)

FORMULA 10 ? IoU-Based K-Means:
```
IoU(box, anchor) = (min(w_b,w_a) * min(h_b,h_a)) / (w_b*h_b + w_a*h_a -
intersection)
d(box, anchor)  = 1 - IoU(box, anchor)   [distance metric]

Algorithm: k=6, random init, iterate until assignments stable
Sort by area: Anchors 1-3 (small) -> Stride-16 | Anchors 4-6 (large) ->
Stride-32
```

8.5 Model-to-Hardware Conversion Pipeline

Step 1 ? Keras to TFLite (convert/convert_tflite.py):
```python
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]   # Post-training quant
tflite_model = converter.convert()   # ~680 KB output
```

Step 2 ? TFLite to kmodel INT8 (convert/ncc_quantize.bat):
```batch
nncase compile --target k210 --dataset calibration_images/
               --inference-type uint8 --input-mean 0 --input-std 255
               yolov3_lite.tflite face.kmodel
```
Output: face.kmodel ~390 KB ? fits K210's 2 MB KPU SRAM

Step 3 ? Deploy:
- SD card: kpu.load("/sd/face.kmodel")
- SPI Flash: kpu.load(0x300000)

8.6 Benchmark ? VISOR vs. Reference Paper (Liu & Xiao, 2022)

Metric	Paper (416x416x3)	VISOR (224x160x1)	Gain
Input pixels	519,168	35,840	14.5x fewer
Input buffer (FP32)	2,076 KB	143 KB	93.1% reduction
Detection scales	3 (strides 8,16,32)	2 (strides 16,32)	Removes 28x20 head
Parameters	~2.30 Million	~398 Thousand	82.7% fewer
Model size (FP32)	23.6 MB	~1.6 MB	93.2% reduction
Quantized (INT8)	~2.5-3.0 MB	~320-400 KB	Fits K210 SRAM
KPU latency	7 ms	3.5 ms	2x faster
Loop FPS	10-12 FPS	28-30 FPS	2.5-3x speedup
Power	0.3 W	0.3 W	Same
Hardware cost	~$30	~$30	Same

FPS Speedup Breakdown:
1. Hardware crop sensor.set_windowing(): 0ms CPU cost
2. 93.1% fewer input pixels: KPU drops from 7ms to 3.5ms
3. Removed stride-8 head: no 28x20 grid computation
4. Fast decode: skip sigmoid/exp for cells with raw objectness <= 0.0
5. Peak memory < 240 KB (K210 has ~1.6 MB spare for other tasks)

---

9. Subsystem 6 ? Cloud Backend (VISOR Server)

File: HELMETAUTHENTICATION/backend/server.js
Runtime: Node.js | **Deploy:** Railway (https://visor-ambulance-backend-production.up.railway.app)

9.1 Dependencies (package.json)

```json
{
  "express":        "^4.19.2",   // HTTP server
  "socket.io":      "^4.7.5",    // WebSocket real-time
  "mqtt":           "^5.7.0",    // MQTT bridge to ESP32
  "mongoose":       "^8.4.1",    // MongoDB ODM
  "firebase-admin": "^12.1.1",   // Firebase Firestore
  "cors":           "^2.8.5",    // CORS middleware
  "dotenv":         "^16.4.5"    // Environment variables
}
```

```

**In-memory state:**
```javascript
const activeRiders     = {};   // vehicle_id -> rider state
const activeAmbulances = {};   // vehicle_id -> ambulance state
const activeAlerts     = {};   // "riderId__ambId" -> alert state
```

9.2 Socket.IO Events and MQTT Topics

Socket.IO Events:

Event	Direction	Purpose
register_vehicle	Client->Server	Register with type (rider/ambulance)
update_location	Client->Server	Stream lat, lng, speed, heading
trigger_emergency	Client->Server	Ambulance activate emergency
cancel_emergency	Client->Server	Ambulance deactivate emergency
proximity_warning	Server->Client	Alert rider of approaching ambulance
ambulance_location	Server->Client	Live ambulance position for HUD
system_state	Server->All	Full state snapshot broadcast
emergency_started	Server->All	Emergency activation announcement
emergency_cancelled	Server->All	Emergency cancellation announcement
emergency_status	Server->Ambulance	Count of warned nearby riders

MQTT:
```
Subscribe: vehicles/+/+/location   (type and ID wildcards)
Publish:   vehicles/rider/{id}/alerts

Alert payload:
{
  "alert":         true,
  "alert_level":   "CRITICAL",
  "distance":      287.4,
  "closing_speed": 40.0,
  "eta":           25,
  "clock":         6,
  "timestamp":     "2026-09-20T15:30:00.000Z"
}
```

9.3 Priority Engine ? Geospatial Algorithms
(backend/src/priority_engine.js)

FORMULA 11 ? Haversine Distance (Great-Circle):
```
phi1, phi2 = latitudes in radians; dlam = lon2 - lon1 in radians
a = sin^2((phi2-phi1)/2) + cos(phi1)*cos(phi2)*sin^2(dlam/2)
d = 2 * R * atan2(sqrt(a), sqrt(1-a))    where R = 6,371,000 m
```

```javascript
function getDistance(lat1, lon1, lat2, lon2) {
    const R = 6371e3;
```
```

```
    const phi1 = toRad(lat1), phi2 = toRad(lat2);
    const a = Math.sin(toRad(lat2-lat1)/2)**2 +
              Math.cos(phi1)*Math.cos(phi2)*Math.sin(toRad(lon2-lon1)/2)**2;
    return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
}
```

FORMULA 12 ? Forward Bearing (Azimuth from North):
```
y = sin(lon2-lon1) * cos(lat2)
x = cos(lat1)*sin(lat2) - sin(lat1)*cos(lat2)*cos(lon2-lon1)
bearing = (atan2(y, x) * 180/pi + 360) mod 360    [0=North, 90=East]
```

FORMULA 13 ? Shortest Angular Difference:
```
diff = (h2 - h1) mod 360
if diff > 180: diff -= 360;   if diff < -180: diff += 360
Result in [-180, +180] degrees
```

FORMULA 14 ? Clock Direction Mapping:
```
hour = round(((angleDiff + 360) mod 360) / 30) mod 12
if hour == 0: hour = 12
Maps bearing offset to 1-12 clock position
```

FORMULA 15 ? Closing Speed:
```
v_close_kmh = max(0, v_ambulance - v_rider)
v_close_ms  = v_close_kmh / 3.6
```

FORMULA 16 ? ETA:
```
ETA_seconds = distance_m / v_close_ms    (only when v_close_ms > 0)
```

FORMULA 17 ? Collision Probability Score (0.0 to 1.0):
```
f_dist  = max(0, 1 - d/500)
f_speed = min(1, v_close/50)    or 0.2 if v_close=0
f_align = max(0, 1 - |headingToRiderAngle|/60)
P_coll  = clamp(0.5*f_dist + 0.3*f_speed + 0.2*f_align, 0, 1)
```

9.4 Ambulance Corridor Filtering Logic

Constants:
```javascript
const ALERT_RADIUS_M = 500;   // Alert riders within 500m
const AHEAD_ANGLE    = 75;    // +/-75 degree forward cone
```

Gate 1 ? Emergency active:

```javascript
if (!ambulance.isEmergency) { clearAlert(); return; }
```

Gate 2 ? Within 500m:
```javascript
if (distance > 500) { clearAlert(); return; }
```

Gate 3 ? Rider AHEAD (in forward cone):
```javascript
const bearing = getBearing(amb.lat, amb.lng, rider.lat, rider.lng);
if (Math.abs(getAngleDifference(amb.heading, bearing)) > 75)
    { clearAlert(); return; }
```

Forward cone visualization:
```
           Ambulance heading H
                  |
       H-75 ------+------ H+75     <- 150 degree cone
              \   |   /
               \  |  /     Riders in this zone get alerts
                \ | /
              [Ambulance]
```

Gate 4 ? Ambulance has NOT passed:
```javascript
const bearingRiderToAmb = getBearing(rider.lat, rider.lng, amb.lat, amb.lng);
if (Math.abs(getAngleDifference(rider.heading, bearingRiderToAmb)) <= 60)
    { clearAlert(); return; }   // Ambulance now in front of rider
```

Alert Level Classification:

Condition	Level
distance <= 150m AND isBehind AND isMovingTowards	CRITICAL
ETA < 15s AND ambulanceFaster	CRITICAL
distance <= 300m AND isBehind AND isMovingTowards AND faster	WARNING
All else	SAFE

9.5 Database ? MongoDB + Firebase Dual Write

MongoDB Schemas (backend/src/database.js):
```javascript
VehicleSchema = { vehicle_id, type, latitude, longitude, speed, heading,
timestamp }
AlertSchema   = { rider_id, ambulance_id, distance, closing_speed, eta,
timestamp, resolved }
```

Dual write on every location update:
```javascript
await Promise.all([
```

```
    VehicleModel.findOneAndUpdate({vehicle_id}, data, {upsert: true}),
    db.collection('vehicles').doc(vehicle_id).set(data, {merge: true})
]);
```

Mock Firestore for local dev: In-memory implementation of same collection/doc API when FIREBASE_SERVICE_ACCOUNT env var is absent.

9.6 REST API Endpoints

Method	Path	Purpose
GET	/health	Server health check
POST	/api/vehicles/location	Update GPS location
GET	/api/vehicles	Get all active vehicles
GET	/api/alerts	Get all active alerts
POST	/api/vehicles/reset	Clear all state

---

10. Subsystem 7 ? Flutter Mobile/Dashboard Application

Directory: HELMETAUTHENTICATION/ | **Flutter SDK:** >=3.10.8 | **App:** v1.0.0

10.1 Architecture ? Provider Pattern

Central state: HelmetProvider (ChangeNotifier)
Orientation: Landscape (EV rider dashboard) | Portrait (ambulance driver)

Key State Fields:

Field	Type	Purpose
_isHelmetWorn	bool	AprilTag vision status (BLE)
_isHeadPresent	bool	FSR physical status (BLE)
_speed	double	Speed km/h (BLE)
_alertLevel	String	SAFE/WARNING/CRITICAL
_ambulanceDistance	double	Distance to ambulance m
_ambulanceEta	int	ETA seconds
_ambulanceClockDirection	int	1-12 clock position
_relativePosAngle	double	Bearing angle -180 to +180
_isEmergencyActive	bool	Ambulance emergency active
_nearbyRiderCount	int	Warned rider count

isAuthenticationApproved:
```dart
bool get isAuthenticationApproved => _isHelmetWorn && _isHeadPresent;
```

Role Selection Flow:
```
RoleSplashScreen
  -> EV RIDER:        Lock landscape -> EvIntroScreen -> EvHomeDashboard
  -> AMBULANCE DRIVER: Lock portrait  -> AmbulanceDriverScreen
```

10.2 BLE Service (lib/services/ble_service.dart)

Service UUID: 12345678-1234-1234-1234-1234567890AB
Helmet UUID: BEB5483E-36E1-4688-B7F5-EA07361B26A8
Strap UUID: CED11C71-7030-4C10-90D1-64582E5507E8

Auto-scan and connect:
```dart
FlutterBluePlus.adapterState.listen((state) {
    if (state == BluetoothAdapterState.on) startAutoScan();
});
// Auto-connect to any device with name containing "ESP32_HELMET"
if (name.contains("ESP32_HELMET")) {
    await FlutterBluePlus.stopScan();
    await connectToDevice(dev);
}
```

Auto-reconnect on disconnect:
```dart
device.connectionState.listen((state) {
    if (state == BluetoothConnectionState.disconnected)
        startAutoScan();   // Immediate restart
});
```

GATT Notify:
```dart
await char.setNotifyValue(true);
char.lastValueStream.listen((value) {
    if (value.isNotEmpty)
        provider.updateHelmetStatus(value.first == 1);  // 0x01 or 0x00
});
```

10.3 Ambulance Tracking Service
(lib/services/ambulance_tracking_service.dart)

Backend: https://visor-ambulance-backend-production.up.railway.app

GPS Stream:
```dart
Geolocator.getPositionStream(
    locationSettings: AndroidSettings(
        accuracy: LocationAccuracy.bestForNavigation,
        distanceFilter: 0,
        intervalDuration: Duration(milliseconds: 500),   // 2 Hz
    ),
)
```

FORMULA 18 ? GPS Speed Conversion:
```
speed_kmh = position.speed_ms * 3.6
```

Exponential Backoff Reconnection:
```dart
_reconnectDelaySec = (_reconnectDelaySec * 2).clamp(2, 32);
// Sequence: 2s -> 4s -> 8s -> 16s -> 32s (capped)
```

Socket.IO WebSocket only (lowest latency):
```dart
io.io(url, io.OptionBuilder().setTransports(['websocket'])
    .disableAutoConnect().setTimeout(10000).build())
```

TTS Voice Alerts (rate-limited 10s minimum):
```dart
final msg = level == 'CRITICAL'
    ? 'Ambulance very close! Pull over immediately!'
    : 'Ambulance approaching from behind. Please give way.';
```

Vibration Patterns:
```dart
if (level == 'CRITICAL')
    Vibration.vibrate(pattern: [0, 200, 100, 200, 100, 500]);
    // 200ms ON, 100ms OFF, 200ms ON, 100ms OFF, 500ms ON
else
    Vibration.vibrate(duration: 400);   // Single 400ms pulse
```

10.4 EV Dashboard Arc Speedometer (lib/screens/ev_home_dashboard.dart)

FORMULA 19 ? Arc Active Sweep:
```
fraction    = clamp(speed / 120.0, 0, 1)    [maxSpeed = 120 km/h]
activeSweep = 240.0 * fraction              [degrees of arc lit]
startAngle  = 210 degrees, sweepAngle = 240 degrees
```

FORMULA 20 ? Tick Angle (13 major ticks):
```
theta_i = 210 + (240/12) * i   degrees,  i = 0..12
```

7-Color Gradient (120 arc segments):
```
Deep green -> Bright green -> Yellow-green -> Yellow -> Orange -> Red-orange
-> Red
```

Speed Animation: 600ms eased tween on value change

HUD Slide-In Animation (alert active):
```dart
Transform.translate(
    offset: Offset(0, (1 - _hudAnim.value) * 60),  // slide up
    child: Opacity(opacity: _hudAnim.value, child: hudWidget)
```

)
```

### 10.5 Ambulance Proximity HUD (lib/screens/ambulance_proximity_hud.dart)

**4 Animation Controllers:**

| Controller | Duration | Purpose |
|---|---|---|
| _pulseCtrl | 1400 ms | Danger rings (repeat+reverse) |
| _scanCtrl | 2200 ms | Scan sweep line (repeat) |
| _shakeCtrl | 120 ms | Screen shake CRITICAL |
| _flashCtrl | 300 ms | Screen flash CRITICAL |

**Color by Alert Level:**

| Level | Primary | Background |
|---|---|---|
| SAFE | #00E5FF Cyan | #000E1A |
| WARNING | #FFAA00 Amber | #1A0C00 |
| CRITICAL | #FF2222 Red | #1A0000 |

**Lane View Custom Painter draws:**
1. Danger rings (4 animated expanding circles around rider)
2. Road surface (gradient perspective strip + dashed center markings)
3. Rider icon (green glowing circle + bike vector symbol)
4. Ambulance icon (alert-color circle + medical cross + warning triangle)
5. Danger line (dashed connecting line rider-to-ambulance)

**FORMULA 21 ? Ambulance Canvas Position:**
```
t = clamp(distance / 500.0, 0, 1)
verticalDist   = t * canvas.height * 0.28   [far=bottom-edge, close=center]
lateralOffset = (riderAngle / 180.0) * 30.0
ambPos = Offset(cx + lateralOffset, cy + verticalDist + 30)
```

**FORMULA 22 ? Distance Lerp (every 80ms):**
```
d_display = d_display + 0.12 * (d_real - d_display)
Gives smooth animated counter
```

**CRITICAL Shake:**
```dart
offset: Offset((_shakeCtrl.value - 0.5) * 4, 0)   // +-2px horizontal shake
```

10.6 Ambulance Driver Interface
(lib/screens/ambulance_driver_screen.dart)

Vehicle ID: AMB_8825508708 (from phone number)

Emergency toggle:
```dart
if (provider.isEmergencyActive) _trackingService.cancelEmergency();
```
```

```
else _trackingService.triggerEmergency();
```

Shows: GPS status, large pulsing emergency button, rider count, live map.

---

11. Subsystem 8 ? ESP32 Rider GPS Tracker

File: HELMETAUTHENTICATION/esp32/esp32_rider_gps/esp32_rider_gps.ino
Hardware: ESP32 + NEO-6M GPS + SSD1306 OLED 128x64 + piezo buzzer

GPS: UART2 GPIO16 RX, GPIO17 TX, 9600 baud ? parsed with TinyGPS++
MQTT Publish: vehicles/rider/RIDER001/location every 1000ms
MQTT Subscribe: vehicles/rider/RIDER001/alerts

Publish format:
```json
{ "vehicle_id":"RIDER001", "latitude":13.0827, "longitude":80.2707,
  "speed":45.2, "heading":180.0, "timestamp":"2026-08-28T15:30:00.000Z" }
```

Alert response:
```c
if (alertLevel == "CRITICAL") tone(BUZZER_PIN, 1000, 100);   // 1000 Hz
if (alertLevel == "WARNING")  tone(BUZZER_PIN, 800, 250);    // 800 Hz
```

OLED display:
```
  SMART TRANSPORT SYS
  [CRITICAL!]
  Ambulance Behind!
  287m @ 6H
```

---

12. AprilTag Generation and Physical Manufacturing

File: generate_apriltag.py
Family: TAG36H11 (robust Hamming error correction)
ID: 0 (0xd5d628584)

10x10 Binary Matrix (1=white, 0=black):
```
[1,1,1,1,1,1,1,1,1,1]   <- outer white border
[1,0,0,0,0,0,0,0,0,1]   <- inner black border
[1,0,1,1,0,1,0,1,0,1]   <- data rows (6x6 payload)
[1,0,0,1,1,1,0,1,0,1]
[1,0,0,1,1,0,0,0,0,1]
[1,0,1,0,1,0,0,0,0,1]
[1,0,0,1,0,1,1,0,0,1]
[1,0,0,0,0,1,0,0,0,1]
[1,0,0,0,0,0,0,0,0,1]   <- inner black border
[1,1,1,1,1,1,1,1,1,1]   <- outer white border
```

```

**FORMULA 23 ? Print Size:**
```
box_size = 50 px/module
size_px = 10 * 50 = 500 x 500 px
size_in = 500 / 300 DPI = 1.667 inches
size_cm = 1.667 * 2.54 = 4.23 cm
```

4.23 cm is optimal for detection at 30-35 cm camera distance.

**Image generation uses NEAREST-neighbor resize (no anti-aliasing blur):**
```python
tag_img = base_img.resize((500, 500), Image.Resampling.NEAREST)
```

---

13. Complete Data Flow and System Integration

13.1 Helmet Authentication Flow (End-to-End)

```
1. Rider puts on helmet
2. K210 OV2640 captures QVGA frame continuously at 28-30 FPS
3. Low-light preprocessing: img.histeq() histogram equalization each frame
4. YOLOv2 KPU inference -> face bounding box (~3.5 ms)
5. EMA smooth bounding box (alpha=0.15), hold MAX_LIFETIME=25 frames on miss
6. Compute eye-region sub-box (top 45% of face height)
7. Compute tight ROI below eye-box for AprilTag (max 120 rows height)
8. find_apriltags(roi=...) -> detect TAG36H11 ID=0, check tcy > face_bottom_y
9. Debounce both: rolling 12-frame window, require 2/12 positives
10. HELMET_WORN = face_seen_stable AND tag_seen
11. State machine: lock PASS or FAIL for 30s, new cycle at 60s
12. K210 UART1 sends: TAG:1 ID:0 X:cx Y:cy\n  (every frame, 115200 baud)
13. Internal ESP32 (Serial2, GPIO3): reads line, parses TAG/ID/X/Y fields
14. Pack into tag_message (20 bytes) -> esp_now_send(receiverMAC, ...)
15. Helmet ESP32: reads FSR (GPIO35) + MQ-3 (GPIO34) every 500ms
16. Runs alcohol hysteresis + FSR 3-state machine
17. Pack into helmet_message (16 bytes) -> esp_now_send(receiverMAC, ...)
18. Receiver ESP32 OnDataRecv: differentiate packets by size
19. SAFE = !linkLost AND !helmetLinkLost AND tag_detected AND fsrAlert AND
!alcoholAlert
20. Motor: analogWrite(MOTOR_ENA, SAFE ? 230 : 100)
21. BLE GATT Notify: helmet=1/0, strap=1/0, speed=46/20
22. Flutter BLE receives -> HelmetProvider updates -> UI rebuilds
```

13.2 Ambulance Alert Flow (End-to-End)

```
1. Ambulance driver opens app -> selects AMBULANCE DRIVER role
2. AmbulanceTrackingService.init() -> Socket.IO connects to Railway backend
3. emit('register_vehicle', {vehicle_id:'AMB_8825508708', type:'ambulance'})
```
```

```
4. GPS stream at 500ms -> emit('update_location', {lat,lng,speed,heading})
each update
5. Driver taps EMERGENCY -> emit('trigger_emergency')
6. Backend: activeAmbulances[id].isEmergency = true
7. Backend evaluates ALL active riders on every ambulance location update:
   a. Haversine distance <= 500m?
   b. Rider in +/-75 degree forward cone of ambulance heading?
   c. Ambulance NOT already passed rider (bearing check)?
   d. evaluatePriority(): ETA, P_collision, alert level
8. For each qualifying rider:
   a. emit('proximity_warning', alertPayload) to rider socket room
   b. mqttClient.publish('vehicles/rider/{id}/alerts', alertJson)
9. Rider Flutter receives proximity_warning:
   a. HelmetProvider._isAmbulanceApproaching = true
   b. _hudCtrl.forward() -> HUD slides in (500ms eased animation)
   c. TTS voice alert (10s minimum rate-limit)
   d. Vibration pattern (CRITICAL: pulse, WARNING: 400ms)
10. ESP32 GPS tracker receives MQTT:
    a. tone(BUZZER_PIN, 1000/800 Hz, 100/250 ms)
    b. Alert LED on, OLED updates
11. As ambulance moves closer:
    a. Backend re-evaluates every location update
    b. Distance updates -> rider receives updated alertPayload
    c. HUD distance counter animates via lerp (alpha=0.12, every 80ms)
    d. Alert may escalate WARNING -> CRITICAL
12. Ambulance passes rider (bearing check: ambulance now in front):
    a. clearAlert() for that rider/ambulance pair
    b. Rider receives {triggerAlert:false, alertLevel:'SAFE'}
    c. _hudCtrl.reverse() -> HUD slides out
13. Ambulance cancels emergency:
    a. emit('cancel_emergency')
    b. All activeAlerts for that ambulance cleared
    c. All warned riders receive clear signal
```

---

14. All Formulas and Mathematical Models (Master Reference)

#	Name	Formula
1	EMA smoothing (face box)	x[t] = alpha*x_raw[t] + (1-alpha)*x[t-1], alpha=0.15/0.30
2	AprilTag ROI bounds	roi_x=max(0,face_left_x-25), roi_w=min(right_x-left_x+50, 320-roi_x), roi_h=min(240-roi_y, 120)
3	Alcohol hysteresis	Enter: >1900, Exit: <1800 (100 count dead-band)
4	FSR voltage divider	V_out = Vcc * R_pull / (R_FSR + R_pull); ADC = 4095 * R_pull / (R_FSR + R_pull)
5	Hall baseline calibration	baseline = (1/20)*SUM(ADC_i), deviation = |ADC-baseline|, strap = deviation<150
6	ReLU6 activation	min(max(0, x), 6)
7	YOLO total loss	5.0*L_coord + L_conf + L_class; noobj_weight=0.5; per scale, sum s16+s32
8	YOLO box encoding	tx=cx*gw-gx, ty=cy*gh-gy, tw=log(w/anchor_w), th=log(h/anchor_h)

9	YOLO box decoding	bx=sig(tx)+col, by=sig(ty)+row, bw=anch_w*exp(tw), bh=anch_h*exp(th)
10	K-means anchor IoU	IoU(b,a)=(min(wb,wa)*min(hb,ha))/(wb*hb+wa*ha-intersect), d=1-IoU
11	Haversine distance	a=sin^2(Dphi/2)+cos(p1)*cos(p2)*sin^2(Dlam/2); d=2*R*atan2(sqrt(a),sqrt(1-a))
12	Forward bearing	y=sin(Dlam)*cos(p2), x=cos(p1)*sin(p2)-sin(p1)*cos(p2)*cos(Dlam), b=(atan2(y,x)*180/pi+360)%360
13	Angular difference	diff=(h2-h1)%360; if>180: -=360; if<-180: +=360
14	Clock direction	hour=round(((diff+360)%360)/30)%12; if hour==0: hour=12
15	Closing speed	v_close=max(0,v_amb-v_rider); v_ms=v_close/3.6
16	ETA	ETA=distance/v_close_ms (only when v_close_ms>0)
17	Collision probability	P=clamp(0.5*(1-d/500)+0.3*min(1,v/50)+0.2*max(0,1-|angle|/60), 0, 1)
18	GPS speed conversion	speed_kmh = speed_ms * 3.6
19	Speedometer arc	fraction=clamp(spd/120,0,1); activeSweep=240*fraction degrees
20	Tick angle	theta_i = 210 + 20*i degrees (i=0..12)
21	Ambulance canvas pos	t=dist/500; vertDist=t*H*0.28; latOff=(angle/180)*30
22	HUD distance lerp	d_display += 0.12*(d_real - d_display) every 80ms
23	AprilTag print size	500px / 300 DPI = 1.667 in = 4.23 cm

---

15. All Key Parameters, Thresholds and Constants

Vision System (K210 / MaixPy)

Parameter	Value	Description
CONF_THRESHOLD	0.18	YOLOv2 minimum detection confidence
NMS_THRESHOLD	0.30	Non-Maximum Suppression IoU threshold
SMOOTHING (alpha)	0.15 / 0.30	EMA smoothing factor (sipeed / deploy)
MAX_LIFETIME	10 / 25	Ghost frames before face expiry
EYE_CROP_RATIO	0.45	Top 45% of face = eye box
ROI_HORIZONTAL_PAD	25 px	Horizontal padding each side of ROI
ROI_MAX_HEIGHT	120 rows	Memory-safe maximum ROI height
HISTORY_LEN	12	Rolling debounce window size
HISTORY_HIT	2	Minimum positives in window for PASS
Camera Resolution	320x240 QVGA	RGB565 input format
TAG_HOLD_FRAMES	15	Frames to hold tag detection state
AXIS_TOLERANCE	1.0	Full-width horizontal check factor

State Machine Timing

Parameter	Value	Description
CYCLE_INTERVAL_MS	60,000 ms	Full detection cycle period
SCAN_WINDOW_MS	30,000 ms	Active scanning phase duration
HEARTBEAT_TIMEOUT_MS	2,000 ms	K210 silence before keepalive

Sensor Thresholds (Helmet ESP32)

Parameter	Value	Description
ALCOHOL_PIN	GPIO 34	MQ-3 analog input
FSR_PIN	GPIO 35	FSR analog input
HALL_PIN	GPIO 32	Hall effect analog input
ALCOHOL_THRESHOLD	1900 ADC	Alert entry threshold
ALCOHOL_HYSTERESIS	100 ADC	Hysteresis dead-band width
FSR_TOUCH_THRESHOLD	80 ADC	Helmet worn entry threshold
FSR_REMOVE_THRESHOLD	25 ADC	Definitive removal threshold
FSR_HOLD_MS	6,000 ms	PASS hold window
FSR_REMOVAL_DEBOUNCE_MS	500 ms	Removal debounce duration
HALL_THRESHOLD	150 ADC	Strap deviation for unlock
MQ3_WARMUP_MS	120,000 ms	Sensor heater warm-up
SEND_INTERVAL	500 ms	ESP-NOW transmission rate

Motor Control (Receiver ESP32)

Parameter	Value	Description
SPEED_LIMITED	100 PWM	~20 km/h restricted speed
SPEED_MAXIMUM	230 PWM	~46 km/h full speed
LINK_TIMEOUT_MS	5,000 ms	ESP-NOW link loss timeout
MOTOR_IN1	GPIO 12	L298N direction pin 1
MOTOR_IN2	GPIO 13	L298N direction pin 2
MOTOR_ENA	GPIO 14	L298N PWM enable

Backend / Cloud Parameters

Parameter	Value	Description
ALERT_RADIUS_M	500 m	Alert radius around ambulance
AHEAD_ANGLE	75 degrees	Forward cone half-angle
PASSED_ANGLE	60 degrees	Angle to detect ambulance passed
CRITICAL_DISTANCE	150 m	Immediate critical threshold
CRITICAL_ETA	15 s	Time-based critical threshold
WARNING_DISTANCE	300 m	Warning alert threshold
pingTimeout	60,000 ms	Socket.IO ping timeout
pingInterval	25,000 ms	Socket.IO keepalive interval
BACKOFF_INITIAL	2 s	Client reconnect initial delay
BACKOFF_MAX	32 s	Client reconnect maximum delay
MQTT_QOS	1	At-least-once delivery

GPS / Tracking Parameters

Parameter	Value	Description
GPS_UPDATE_MS	500 ms	Android GPS polling interval
MQTT_PUBLISH_MS	1,000 ms	ESP32 GPS publish rate
DISTANCE_FILTER	0 m	Update on any position change
GPS_ACCURACY	bestForNavigation	Android accuracy mode

---

16. Hardware Bill of Materials (BOM)

| Component | Specification | Qty | Role |

|---|---|---|---|
Sipeed Maixduino	K210 dual RISC-V + OV2640 + ESP32	1	Edge AI vision inference
Receiver ESP32	ESP32-WROOM-32	1	BLE hub + motor control
Helmet ESP32	ESP32-WROOM-32	1	Sensor acquisition
MQ-3 Alcohol Sensor	Analog output, 5V heater, electrochemical	1	Breath alcohol detection
FSR Pressure Sensor	Force-sensitive resistor, analog	1	Head presence detection
Hall Effect Sensor	Analog hall (A3144 or similar)	1	Strap lock detection
L298N Motor Driver	Dual H-bridge, up to 2A	1	EV motor speed control
BO Gearbox Motor	DC brushed motor with gearbox	1	Vehicle wheel drive
NEO-6M GPS Module	UART 9600 baud, NMEA sentences	1	Rider GPS tracker
SSD1306 OLED Display	128x64 px, I2C, address 0x3C	1	Alert display (tracker)
Piezo Buzzer (active)	5V, 85 dB	2	K210 alert + tracker
RGB LED set	Red GPIO12, Green GPIO13, Blue GPIO14	3	Status indicators
10 kOhm Resistor	Pull-down for FSR voltage divider	1	FSR circuit
MicroSD Card	>= 4 GB	1	Store face.kmodel
Motorcycle Helmet	Commercial full-face or half-face	1	AprilTag mounting platform
AprilTag Sticker	4.23 cm, printed 300 DPI, TAG36H11 ID 0	1	Vision interlock fiducial

---

17. Software Dependencies and Versions

Flutter Application (pubspec.yaml)

Package	Version	Purpose
flutter_blue_plus	^2.3.0	BLE scanning and GATT
provider	^6.1.5+1	State management
google_fonts	^8.0.1	Orbitron, Rajdhani typefaces
flutter_animate	^4.5.2	Declarative animations
permission_handler	^11.3.1	Runtime permissions
geolocator	^10.1.0	GPS streaming
socket_io_client	^2.0.3+1	Socket.IO WebSocket client
flutter_tts	^3.8.5	Text-to-speech
vibration	^3.2.0	Haptic vibration patterns
google_maps_flutter	^2.6.0	Map rendering
http	^1.6.0	HTTP REST client
model_viewer_plus	^1.7.0	3D model viewer
google_places_flutter	^2.1.1	Places autocomplete

Node.js Backend (package.json)

Package	Version	Purpose
express	^4.19.2	HTTP server framework
socket.io	^4.7.5	Real-time WebSocket server

mqtt	^5.7.0	MQTT broker bridge client
mongoose	^8.4.1	MongoDB ODM
firebase-admin	^12.1.1	Firebase Admin SDK
cors	^2.8.5	CORS middleware
dotenv	^16.4.5	Environment variable management

Python ML Pipeline

Package	Purpose
tensorflow >= 2.x	Model definition, training, TFLite conversion
numpy	Array operations, K-means anchor clustering
Pillow (PIL)	AprilTag PNG image generation
nncase (NCC compiler)	TFLite to kmodel INT8 quantization

ESP32 Arduino Libraries

Library	Purpose
esp_now.h	ESP-NOW connectionless protocol
WiFi.h	WiFi adapter management
esp_wifi.h	Low-level channel control
BLEDevice.h / BLEServer.h	BLE GATT server creation
BLE2902.h	GATT descriptor for notifications
TinyGPS++	NMEA GPS sentence parsing
PubSubClient	MQTT client
ArduinoJson	JSON parsing and serialization
Adafruit_SSD1306	OLED display driver
HardwareSerial	UART communication

MaixPy K210 Runtime Modules

Module	Purpose
sensor	OV2640 camera control and configuration
image	Frame capture, AprilTag detection, drawing primitives
KPU	Neural network hardware inference acceleration
lcd	ILI9341 TFT display output
FPIOA / GPIO	Pin multiplexing and digital I/O
UART	Serial communication to ESP32
gc	Memory garbage collection (critical for SRAM management)
time / machine	Timing utilities and hardware control

---

18. Performance Summary and Test Results

Vision System (K210)

Metric	Result
KPU inference latency (224x160x1)	3.5 ms
Full loop frame rate	28-30 FPS
EMA smoothing factor	0.15 (sipeed) / 0.30 (deploy)
Ghost frame persistence	10-25 frames
Typical AprilTag ROI	~80x100 pixels

Peak memory at inference	< 240 KB
face.kmodel file size	~390 KB (INT8)
K210 KPU SRAM used	< 400 KB of 2 MB
Speedup vs. reference paper	2.5x-3.0x FPS gain

Wireless Latency

Link	Typical Latency	Range
K210 to Internal ESP32 UART	< 1 ms	On-PCB
ESP-NOW Internal to Receiver	< 5 ms	~100 m LoS
ESP-NOW Helmet to Receiver	< 5 ms	~100 m LoS
BLE GATT Notify	15-30 ms	~10 m
Socket.IO on 4G LTE	20-80 ms	Internet

System End-to-End Alert Latency

Stage	Time
GPS acquisition to app	~500 ms (GPS update interval)
App to Server (Socket.IO)	~50-100 ms
Server compute + emit	< 1 ms
Server to Rider (Socket.IO)	~50-100 ms
App processes + TTS start	~200 ms
Total estimated	**800 ms - 1.5 seconds**

---

19. Novel Contributions and Patent Claims

Claim 1 ? Multi-Factor Visual-Physical Helmet Interlock

A system for verifying correct helmet usage on an electric vehicle operator, comprising:

(a) An embedded neural network processor (K210) running a YOLOv2-based model to detect the operator's face via an OV2640 camera.

(b) A visual fiducial marker of family TAG36H11 with ID 0, physically affixed to the outer surface of the helmet at chin or visor level.

(c) A detection algorithm running on said processor that accepts helmet as WORN only when simultaneously: (i) a face bounding box is detected and stable over a rolling history window, AND (ii) a TAG36H11 fiducial marker is detected within a region geometrically below the eye-region portion of said face bounding box.

(d) A force-sensitive resistor (FSR) embedded in the helmet interior, confirming physical head contact by exceeding a pressure threshold (ADC > 80).

(e) An alcohol sensor (MQ-3) mounted in the helmet, blocking operation when ethanol vapor exceeds a configured threshold.

(f) A motor interlock that restricts the electric vehicle to approximately 20 km/h when any single condition fails.

NOVELTY STATEMENT: Prior art (vision-only or RFID-only) can be bypassed by holding a helmet near a camera or holding an RFID tag near a reader. This invention requires simultaneous confirmation from three independent sensing modalities (visual geometry, physical pressure, chemical), making bypass attacks against any single sensor ineffective.

Claim 2 ? Geometric Fiducial Relative Position Interlock

A method for helmet compliance authentication comprising:

(a) Detecting a face bounding box using a convolutional neural network.
(b) Computing an eye-region sub-box covering the top 45% of face height.
(c) Constructing a region-of-interest (ROI) immediately below the eye-region boundary, bounded to stay within available SRAM (maximum 120 pixel rows).
(d) Accepting the helmet as WORN only when a fiducial marker (AprilTag ID 0) is detected whose center Y-coordinate exceeds the eye-region boundary Y-coordinate.

This geometric relative-position constraint ensures the fiducial is at the chin/visor position and cannot be spoofed by placing the tag anywhere else in the camera's field of view.

Claim 3 ? Ahead-Only Ambulance Proximity Alert with Directional Filtering

A system for alerting electric vehicle riders ahead of an approaching emergency vehicle comprising:

(a) A cloud server receiving real-time GPS coordinates, speed, and heading from both ambulances and EV riders via WebSocket connections.
(b) A forward-corridor filter that computes the bearing from the ambulance to each rider and suppresses alerts for riders outside a configurable forward cone (default: +/-75 degrees of ambulance heading).
(c) A pass-detection algorithm that additionally suppresses alerts for any rider for whom the ambulance has already overtaken (bearing from rider to ambulance within +/-60 degrees of rider heading).
(d) Alert delivery through Socket.IO (visual HUD), MQTT (hardware buzzer), text-to-speech voice, and haptic vibration, ensuring alert receipt despite helmet insulation.
(e) Automatic alert clearance when pass-detection or radius conditions are no longer met.

NOVELTY: Prior art broadcasts alerts to all vehicles in a radius, causing false alarms for vehicles traveling in the same direction away from the ambulance, or already passed. This invention provides directional geometric filtering.

Claim 4 ? Memory-Constrained Fiducial Detection via Dynamic ROI

A method for AprilTag detection on SRAM-limited embedded hardware comprising:

(a) Computing a search ROI dynamically based on the current face detection bounding box.

(b) Constraining ROI pixel area such that the internal allocation of the tag detector remains below available SRAM (formula: roi_w * roi_h * 12 < SRAM_limit).
(c) Limiting ROI height to a configurable maximum (120 rows).
(d) Running memory garbage collection before each detection call.

This enables full TAG36H11 detection on a platform (K210 with 2 MB SRAM) that would fail on a full-resolution search.

Claim 5 ? Alcohol-Integrated Motor Speed Interlock with Hysteresis

A helmet-integrated alcohol detection and vehicle speed restriction system comprising:

(a) An MQ-3 electrochemical sensor with a 120-second firmware-gated warmup period.
(b) A hysteresis comparator with configurable enter threshold (1900 ADC) and exit threshold (1800 ADC) preventing oscillation at the boundary.
(c) An ESP-NOW wireless link transmitting sensor state to the vehicle controller with maximum 10 ms latency.
(d) Motor PWM modulation restricting maximum vehicle speed when alcohol is detected.

Claim 6 ? FSR Debounced Helmet Occupancy State Machine

A helmet occupancy detection system comprising:

(a) A force-sensitive resistor in a voltage-divider circuit embedded in the helmet.
(b) A three-state machine (IDLE, PASS_HOLD, FAIL) with a configurable 6-second hold window after initial contact.
(c) A sustained-removal debounce requiring sub-threshold pressure for 500 ms continuously before declaring helmet removal.
(d) Dual-threshold hysteresis design (entry threshold 80 >> removal threshold 25) preventing false transitions during riding vibration.

---

20. Conclusion and Future Work

Summary

VISOR achieves a complete, end-to-end smart transportation safety system:

- **Edge AI at 28-30 FPS** on a 0.3 W, $30 microcontroller
- **Multi-factor helmet authentication** eliminating all single-sensor bypass attacks
- **Sub-second cloud alert latency** for ambulance proximity warning
- **Multi-modal alerts** (HUD + TTS + vibration + hardware buzzer) ensuring no alert is missed
- **Directional geometric filtering** targeting only relevant vehicles in ambulance path
- **Motor speed interlock** requiring vision + pressure + sobriety simultaneously

Future Work

1. **Drowsiness Detection:** Extend K210 vision to detect eye closure using same camera pipeline
2. **V2X Integration:** Replace phone GPS with DSRC/C-V2X modules for sub-100ms direct vehicle communication
3. **Traffic Signal Preemption:** Backend API adapter for traffic controller green-signal priority when ambulance detected
4. **Multi-Tag Helmet Verification:** Multiple AprilTags at different positions to verify complete fastening
5. **Federated Learning:** Anonymized edge event collection for continuous model improvement without raw images
6. **Solar-Powered Helmet ESP32:** Photovoltaic cell on helmet exterior for indefinite sensor power
7. **CAD System Integration:** API adapter for city-level Computer-Aided Dispatch ambulance tracking feeds
8. **Indian Rider Dataset:** Retrain YOLOv3-Lite on India-specific rider data for improved local accuracy

---

21. Appendices

Appendix A ? Complete File Structure

```
VISOR MAIN PROJECT FOLDER/
+-- boot.py                              <- AprilTag-only standalone detection
+-- generate_apriltag.py                 <- TAG36H11 ID 0 image generator (Pillow)
+-- helmet_apriltag.png                  <- 500x500 AprilTag output image
+-- sipeed/
|   +-- main.py                          <- K210 vision + 60s state machine + UART
+-- internal_esp32/
|   +-- internal_esp32.ino               <- K210 UART parser + ESP-NOW sender
+-- receiveresp/
|   +-- receiveresp.ino                  <- ESP-NOW hub + BLE GATT + L298N motor
+-- helmet_esp32/
|   +-- helmetesp/
|       +-- helmetesp.ino                <- MQ-3 + FSR + Hall + ESP-NOW sender
+-- FACE Detection (eye) + AprilaTag Detection/
|   +-- model/
|   |   +-- yolov3_lite.py             <- MobileNetV2-Lite YOLO model architecture
|   |   +-- train.py                   <- Custom YOLO loss + training pipeline
|   |   +-- kmeans_anchors.py          <- IoU K-means anchor clustering
|   |   +-- anchors.txt                <- Computed anchor values for 224x160
|   +-- convert/
|   |   +-- convert_tflite.py          <- Keras H5 -> TFLite converter
|   |   +-- ncc_quantize.bat           <- TFLite -> kmodel INT8 (NCC compiler)
|   +-- deploy/
|   |   +-- main.py                    <- Production K210 inference script
```

```
|   |   +-- boot_nn.py                <- Alt boot with NN detection
|   +-- benchmark/
|       +-- benchmark_report.md        <- Performance vs. reference paper
+-- HELMETAUTHENTICATION/
    +-- lib/
    |   +-- main.dart                   <- Flutter app entry point + theme
    |   +-- providers/
    |   |   +-- helmet_provider.dart    <- Central ChangeNotifier state store
    |   +-- services/
    |   |   +-- ble_service.dart        <- BLE auto-scan + GATT notify client
    |   |   +-- ambulance_tracking_service.dart <- GPS + Socket.IO + TTS +
haptics
    |   +-- screens/
    |       +-- role_splash_screen.dart  <- Role selection (EV Rider /
Ambulance)
    |       +-- ev_intro_screen.dart     <- Intro animation sequence
    |       +-- ev_home_dashboard.dart   <- Main dashboard + arc speedometer
    |       +-- ambulance_proximity_hud.dart <- Cinematic 4-controller alert
HUD
    |       +-- ambulance_driver_screen.dart <- Ambulance emergency interface
    |       +-- ambulance_map_screen.dart    <- Live map view
    |       +-- splash_screen.dart           <- Loading screen
    +-- backend/
    |   +-- server.js                   <- Express + Socket.IO + MQTT server
    |   +-- src/
    |   |   +-- database.js             <- MongoDB + Firebase + mock setup
    |   |   +-- priority_engine.js      <- Haversine + bearing + alert logic
    |   +-- package.json                <- Node.js dependencies
    |   +-- railway.json                <- Railway cloud deployment config
    +-- esp32/
    |   +-- esp32_rider_gps/
    |       +-- esp32_rider_gps.ino    <- GPS + MQTT + OLED + buzzer tracker
    +-- pubspec.yaml                    <- Flutter package dependencies
+-- SIH/
    +-- THE_VISOR_SIH2026_Presentation.pptx  <- SIH 2026 presentation deck
```

Appendix B ? BLE UUID Reference

Characteristic	UUID
Service	12345678-1234-1234-1234-1234567890AB
Helmet Vision	BEB5483E-36E1-4688-B7F5-EA07361B26A8
Strap FSR	CED11C71-7030-4C10-90D1-64582E5507E8
Speed	DFB5483E-36E1-4688-B7F5-EA07361B26A8

Appendix C ? ESP-NOW MAC Addresses

Device	Role	MAC Address
Receiver ESP32	Central hub target	28:05:A5:E2:C6:64
Helmet ESP32 target	Configured in helmetesp.ino	A0:B7:65:47:67:7C

Note: Verify actual device MACs before deployment. Use WiFi.macAddress() in setup() to print.

Appendix D ? Backend Environment Variables

Variable	Purpose
PORT	HTTP server port (auto-set by Railway)
MONGODB_URI	MongoDB Atlas connection string
FIREBASE_SERVICE_ACCOUNT	Firebase service account JSON (stringified)
FIREBASE_DATABASE_URL	Firebase Realtime Database URL
MQTT_BROKER_URL	MQTT broker (default: mqtt://broker.hivemq.com:1883)

Appendix E ? YOLOv2 Anchor Values (deploy/main.py)

```python
ANCHORS = (1.889, 2.5245,      # Anchor 1: 1.89 x 2.52 grid units
           2.9465, 3.94056,     # Anchor 2: 2.95 x 3.94 grid units
           3.99987, 5.3658,     # Anchor 3: 4.00 x 5.37 grid units
           5.155437, 6.92275,   # Anchor 4: 5.16 x 6.92 grid units
           6.718375, 9.01025)   # Anchor 5: 6.72 x 9.01 grid units
```

These 5 anchors were determined by K-means clustering (1-IoU distance) on the face training dataset at 320x240 input resolution.

Appendix F ? GPIO Pin Assignment Summary

Sipeed Maixduino (K210 side):

MaixPy Pin	Function	Connection
IO24	GPIOHS0	Buzzer
IO12	GPIOHS1	Red LED
IO13	GPIOHS2	Green LED
IO14	GPIOHS3	Blue LED
IO6	UART1_RX	From ESP32 GPIO1 (ACK)
IO7	UART1_TX	To ESP32 GPIO3 (data)

Helmet ESP32:

GPIO	Sensor	ADC Channel
GPIO 32	Hall Effect	ADC1_CH4
GPIO 34	MQ-3 Alcohol	ADC1_CH6
GPIO 35	FSR Pressure	ADC1_CH7

Receiver ESP32:

GPIO	Component	Purpose
GPIO 12	L298N IN1	Motor direction 1
GPIO 13	L298N IN2	Motor direction 2
GPIO 14	L298N ENA	Motor PWM speed

VISOR Complete Technical Project Report ? End of Document
Version 2.0 | September 2026

All source code analyzed line-by-line | 23 Mathematical Formulas | 6 Patent Claims
Prepared for: Conference/Journal Paper Submission and Patent Filing